



DevTracker

Summary

1.0 Introduction

1.1 Objective

2.0 MVP

2.1 Requirements

2.2 Stacks

Introduction

DevTracker is a software focused on developers, your functions are to make study sessions, make tasks (to do list), graph goals and see your evolution in programming.

Objective

The objective is to take the evolution of users for they're, make your lives easier. Also these, automatizing the analytics for users, connecting to GitHub and looking at the metrics, commits, repositories and more.

MVP

- Register study hours
- Manage tasks and goals
- Track progress in technologies
- Visualize dashboards and evolution metrics
- Track consistency and streaks
- GitHub API integration for activity data

Requirements

Functional Requirements

FR1 (User Auth)

The system should allow users to:

- Create account
- Log in
- Log out

FR2 (Profile management)

The system should allow the user to:

- See your profile
- Edit basic data
- Track personal metrics

FR3 (Study hours log)

The system should allow you to register:

- Study hours
- Date of session
- Optional description

FR4 (Task management)

The system should allow:

- Create tasks
- Edit tasks
- Complete tasks
- Exclude tasks

FR5 (Goals management)

The system should allow for setting goals related to studies and technical development.

FR6 (Technology monitoring)

The system should allow:

- Add technologies
- Track progress
- View percentage change

FR7 (Metrics Dashboard)

The system need to present:

- Study hours
- General progress
- Done tasks
- Actually streak
- Evolution metrics

FR8 (Consistency system)

The system should track the sequence of consecutive days of user activity.

FR9 (GitHub integration)

The system should allow connecting a GitHub account for collecting public data.

FR10 (GitHub activity view)

The system should display:

- Recent activity
- Repositories
- Languages used
- Frequency of activity

Non-Functional Requirements (NFRs)

NFR1 (Performance)

The system must respond to user requests in a timely manner.

NFR2 (Scalability)

The architecture should allow for future growth without the need for complete restructuring.

NFR3 (Security)

The system must:

- Use JWT authentication.
- Protect private routes
- Store encrypted passwords

NFR4 (Responsiveness)

The interface should function correctly in:

- Desktops
- Tablets
- Mobile

NFR5 (Usability)

The system must have an interface:

- Modern
- Intuitive
- Minimalist

NFR6 (Data persistence)

The data should be stored in a relational database using PostgreSQL.

NFR7 (Web compatibility)

The system should work on all major modern web browsers.

NFR8 (Maintainability)

The system should allow for organized maintenance and future upgrades.

NFR9 (Availability)

The system must be accessible online via web deployment.

NFR10 (Version control)

The project should utilize:

- Git
- GitHub

Stacks

Front-end

- React
- Tailwind CSS
- Recharts
- Lucide
- Framer motion

Back-end

- Java
- Spring Boot
- Spring Data
- Spring Security
- JWT
- JUnit
- JPA/Hibernate
- Maven
- PostgreSQL

Tools

- Git
- Github
- Postman
- Vercel
- Render
- Swagger